

# MANUALE TECNICO

## SOMMARIO

### 1. Introduzione

- 1.1 Scopo del documento
- 1.2 Ambito del progetto
- 1.3 Destinatari

### 2. Panoramica dell'architettura

- 2.1 Diagramma ad alto livello
- 2.2 Componenti principali
  - 2.2.1 Frontend Next.js
  - 2.2.2 API REST (Next API Routes + Prisma)
  - 2.2.3 Database PostgreSQL
  - 2.2.4 Stato di autenticazione (cookie JWT)
  - 2.2.5 Algoritmi usati

### 3. Requisiti di sistema

- 3.1 Software di base

Nota: per lavorare in locale è consigliabile usare nvm per gestire versioni multiple di Node.js.

- 3.2 Dipendenze di progetto

- Framework e strumenti principali
- Autenticazione e sicurezza
- UI, animazioni e utilità React
- HTTP client e helpers
- Dev-Dependencies aggiuntive

### 4. Installazione e configurazione

- 4.1 Clonazione del repository
- 4.2 Variabili d'ambiente (.env)
- 4.3 Installazione dipendenze
- 4.4 Inizializzazione Prisma e migrazioni
- 4.5 Generazione CSS con Tailwind

### 5. Struttura del progetto

- 5.1 Cartelle e file chiave
- 5.2 Convenzioni di naming e organizzazione

### 6. Database

- 6.1 Modello dati (schema Prisma)
- 6.2 Script di seed
- 6.3 Migrazioni

### 7. Autenticazione e autorizzazione

- .1 Che cos'è NextAuth.js
- .2 Perché non è stato usato NextAuth, ma una soluzione custom

- 7.1 Flusso di login standard vs. master-admin
  - 7.2 Cookie JWT e setLoginSession
  - 7.3 Middleware authorizeRole
  - 7.4 Protezione delle rotte (authorizeAdmin vs. authorizeRole)
- 8. API Endpoints
  - 8.1 POST /api/auth/login
  - 8.2 POST /api/auth/logout
  - 8.3 GET /api/auth/me
  - 8.4 GET|PUT|DELETE /api/auth/users[:id]
  - 8.5 GET|POST|PUT|DELETE /api/product[:id]
  - 8.6 Esempi di request/response
- 9. Frontend
  - 9.1 Layout generale (components/Layout.tsx)
  - 9.2 SidebarMenu e comportamento “ospite vs. admin”
  - 9.3 ScrollNav e navigazione a “bullet-point”
  - 9.4 Pagine pubbliche vs. protette vs. /admin
  - 9.5 Admin Panel: gestione utenti e prodotti
  - 9.6 Login Page (utente e admin)
- 10. Styling
  - 10.1 Configurazione Tailwind
  - 10.2 Utility personalizzate
  - 10.3 Theme e design system
  - 10.4 Funzioni custom
  - 10.2.5 Funzioni e librerie custom
    - Prisma (prisma.user.\*, prisma.product.\*)
    - AXIOS
- 11. Automazioni e script
  - 11.1 Script NPM (dev, build, watch:css, ...)
  - 11.2 Automazioni eventuali (reminder, deploy, ...)
- 12. Testing & Quality Assurance
  - 12.1 Tipi di test
  - 12.2 Strumenti consigliati
  - 12.3 Best practices
- 13. Deployment
  - 13.1 Impostazione dell’ambiente di produzione
  - 13.2 Piattaforme di hosting
  - 13.3 Variabili d’ambiente in produzione
  - 13.4 Backup del database
- 14. Manutenzione e troubleshooting
  - 14.1 Errori comuni
  - 14.2 Log e debug
  - 14.3 Aggiornamenti di dipendenze
- 15. Appendici
  - 15.1 Glossario

15.2 Risorse e link utili

15.3 Contatti del team

*(ogni elemento del sommario se cliccato reindirizza a quella sezione e/o argomento)*

# 1. Introduzione

La presente sezione fornisce un'inquadratura generale del manuale tecnico di "Artigianato Online", spiegando perché è stato redatto, quali confini di progetto copre e a chi è rivolto.

---

## 1.1 Scopo del documento

Questo manuale tecnico ha l'obiettivo di documentare in modo esaustivo l'architettura, l'installazione, la configurazione e l'utilizzo del sistema "Artigianato Online". In particolare:

- **Chiarire le scelte architetturali** adottate nella realizzazione del frontend (Next.js + Tailwind CSS), del backend (API routes Next.js + Prisma) e del database (PostgreSQL).
- **Guidare gli sviluppatori** nell'onboarding al codice sorgente, mostrando struttura delle cartelle, convenzioni di naming e dipendenze critiche (ad es. [tailwind-scrollbar-hide](#), [clsx](#), [framer-motion](#)).
- **Descrivere i flussi di autenticazione**, con la distinzione fra login standard (customer/artisan) e login master-admin, illustrando l'uso dei cookie JWT tramite la funzione [setLoginSession](#).
- **Illustrare l'uso dell'Admin Panel**, con i dettagli sul CRUD di utenti e prodotti attraverso chiamate axios ([/api/auth/users](#), [/api/product](#)) e le animazioni di apertura/chiusura delle schede.
- **Fornire soluzioni di troubleshooting** per problemi comuni emersi nello sviluppo (ad esempio: scroll "a scatti" nelle sezioni hero/scaffali, errori nella rigenerazione Tailwind CSS, configurazione del file [package.json](#)).

In sintesi, il documento vuole essere un riferimento unico per chiunque debba lavorare o mantenere "Artigianato Online", riducendo tempi di onboarding e facilitando miglioramenti futuri.

---

## 1.2 Ambito del progetto

"Artigianato Online" è una piattaforma web full-stack pensata per consentire a utenti (customer o artisan) e amministratori di:

## 1. Registrarsi e autenticarsi

- Utenti standard/artisan si registrano con email, password crittografata (bcryptjs) e accedono alle pagine protette.
- Admin accede con una password master (12345Aa!) e può gestire utenti/prodotti senza essere trattato come customer/artisan.

## 2. Navigare e acquistare prodotti

- Frontend responsive realizzato in Next.js, con sezioni “hero” e “scaffali” di categorie, animazioni fade-in e scroll-snap tra schermo intero.
- Aggiunta al carrello tramite contesto React (CartContext), API REST /api/cart.

## 3. Gestire dati (Admin Panel)

- CRUD completo di utenti e prodotti: chiamate axios, modali di editing inline con placeholder dinamici, bottoni +/- per prezzo, pulsanti “Elimina” e transizioni animate.
- Dashboard protetta da middleware authorizeAdmin (senza ruolo “ADMIN” nel DB) che verifica solo presenza di sessione valida.

## 4. Stilizzazione e design

- Utilizzo di Tailwind CSS con configurazione personalizzata (globals.css, tailwind.config.js), plugin per scrollbar nascosta e animazioni custom (@keyframes fadeInUp).
- Componenti UI riutilizzabili in /components/ui (SidebarMenu, ScrollNav, Shelves, HeroSection).

## 5. Nuove tecnologie

- TypeScript è un superset di JavaScript che aggiunge tipizzazione statica opzionale e controllo a compile-time. Nel nostro progetto consente di evitare errori a runtime e di avere editor con autocompletamento e refactoring più sicuri. Tutti i file .ts e .tsx ne sfruttano i vantaggi.
- La tipizzazione statica di TypeScript riduce errori a runtime, migliora la manutenibilità e l'esperienza di sviluppo (autocomplete, refactoring).

Rientrano però **al di fuori** di questo manuale: infrastruttura CI/CD, monitoraggio in produzione (es. logging avanzato), documentazione utente finale, e piani di marketing.

---

## 1.3 Destinatari

Questo documento è destinato ai seguenti profili:

- **Sviluppatori Frontend**  
Hanno bisogno di comprendere la struttura delle pagine Next.js, il contesto di autenticazione ([AuthContext](#)), le animazioni di scroll, le utility Tailwind e come aggiungere nuove sezioni “scaffale”.
- **Sviluppatori Backend / DevOps**  
Devono conoscere la configurazione Prisma (schema, migrazioni, seed), le API Routes ([/api/auth](#), [/api/product](#), [/api/orders](#)), la gestione del cookie JWT, e i middleware di autorizzazione.
- **Team QA / Test**  
Necessitano di sapere quali flussi testare (login master-admin, CRUD utenti/prodotti, scroll nav), quali errori comuni verificare in console (404 su [/api/auth/\\_log](#), “scroll a scatti”) e come riprodurre il rebuild Tailwind CSS.
- **Team di manutenzione / Supporto**  
Per risolvere incidenti in produzione, interpretare log di Next.js, correggere problemi di configurazione (es. comando `npm run watch:css` non riconosciuto) e aggiornare dipendenze (Next.js, Prisma, librerie UI).

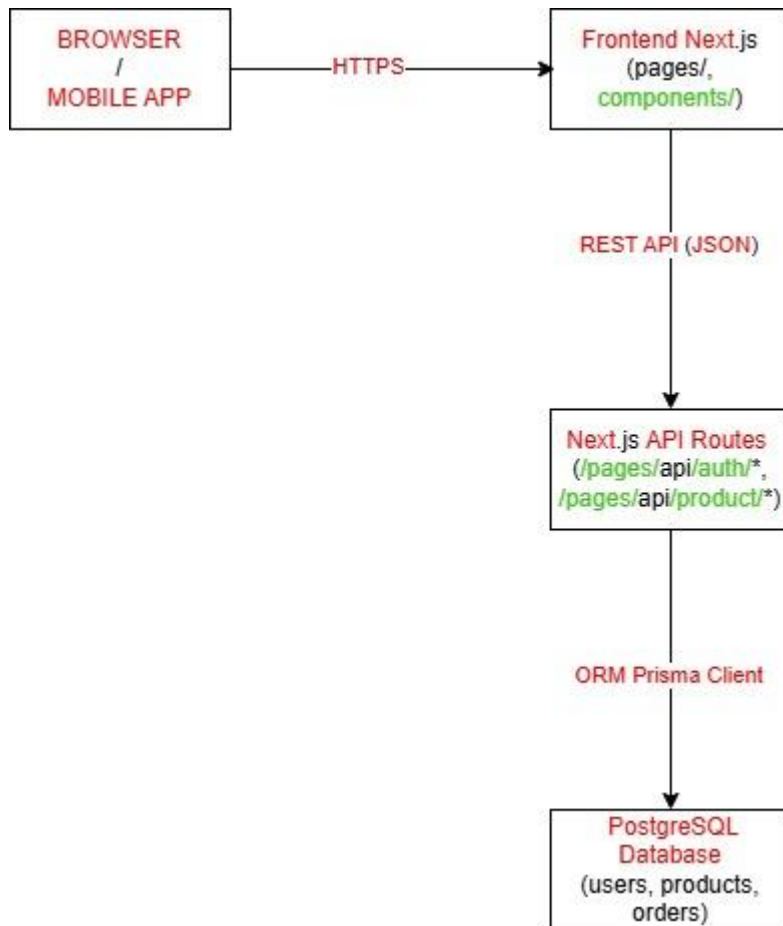
Ogni sezione di questo manuale è quindi calibrata per fornire ai destinatari le informazioni puntuali di cui hanno bisogno, evitando ridondanze e garantendo chiarezza nei passaggi operativi.

## 2. Panoramica dell’architettura

In questa sezione descriviamo l’architettura complessiva di **Artigianato Online**, evidenziando i quattro macro-componenti coinvolti e come interagiscono fra loro.

---

### 2.1 Diagramma ad alto livello



— Il **cookie** “auth” contenente un JWT (firmato da `utils/auth.ts`) passa in ogni richiesta

— Middleware lato API (`authorizeRole`, `authorizeAdmin`) verifica il token e il `payload.role`

---

## 2.2 Componenti principali

### 2.2.1 Frontend [Next.js](#)

**Next.js** fornisce rendering lato server (SSR), generazione statica (SSG), routing basato su filesystem e API Routes integrate. Nel progetto alimenta sia le pagine React (`.tsx`) sia le API (`.ts`) in `/pages/api/*`.

- **.ts** sono file TypeScript che contengono logica back-end o utilità (es. API Routes, utils).
- **.tsx** sono file TypeScript con JSX, usati per componenti React (il frontend vero e proprio).

Nel progetto, tutte le API (login, prodotti, utenti) risiedono in file `.ts` sotto `/pages/api`. Le pagine e i componenti UI (login, admin panel, layout) sono `.tsx` in `/pages` e `/components`.

- **Pages**

- `pages/index.tsx`, `pages/auth/index.tsx` → homepage pubblica e autenticata
- `pages/auth/login.tsx` / `pages/auth/register.tsx` → form di login/registrazione
- `pages/admin/index.tsx` → dashboard di amministrazione (CRUD inline)

- **Components/UI**

- `components/Layout.tsx` → struttura con header, sidebar (non-logged vs admin), scroll-snap “hero/scaffali”
- `components/ui/ScrollNav.tsx` → bullet-point fixed con Intersection Observer
- `components/ui/HeroSection.tsx`, `ShelfSection.tsx`, `AuthProductCard.tsx` → sezione “scaffali”

- **Stilizzazione**

- **Tailwind CSS** con configurazione in `tailwind.config.js` e animazioni custom in `styles/globals.css`. **Tailwind CSS** è un framework utility-first: offre classi pronte (es. `bg-red-600`, `p-4`) per costruire rapidamente UI senza scrivere CSS custom, che permette prototipazione e coerenza stilistica con classi atomiche, riducendo fogli di stile personalizzati e accelerando il workflow
- Plugin `tailwind-scrollbar-hide` per nascondere scrollbar solo visivamente

### 2.2.2 API REST (Next API Routes + Prisma)

Prisma fornisce un client fortemente tipizzato, genera migrazioni e semplifica la scrittura di query SQL, migliorando sicurezza e manutenibilità del codice DB.

- **Autenticazione** (`/pages/api/auth`)

- `login.ts` → riceve `{ email, password[, role] }`, controlla `MASTER_PASSWORD` per admin o bcrypt per user, invoca `setLoginSession(res, payload)`
- `me.ts` → legge cookie JWT, restituisce `{ user }` per sincronizzare il contesto React

- `logout.ts` → pulisce cookie
- `users.ts` → endpoint protetto da `authorizeRole('ADMIN', handler)` per lista utenti
- **Prodotti & Utenti** (`/pages/api/product/[id].ts`, `/pages/api/auth/users/[id].ts`)
  - GET/PUT/DELETE per singolo record tramite Prisma Client (`prisma.product.update`, `prisma.user.delete`)
  - List endpoints (`findMany`, `select`)

### 2.2.3 Database PostgreSQL

- **Schema Prisma** (`schema.prisma` in radice)
  - Modelli: `User`, `Product`, `Category`, `Order`, `OrderItem`, `CartItem`
  - Enum: `Role { CUSTOMER, ARTISAN }`, `OrderStatus`
  - Mappature `@@map("users")`, `@@map("products")` per tabelle legacy
- **Seed e Migrazioni**
  - Comandi `prisma migrate dev` e `prisma db seed` definiti in `package.json`
  - Dati di test iniziali inseriti in `prisma/seed.ts`

### 2.2.4 Stato di autenticazione (cookie JWT)

- **Creazione sessione**
  - `utils/auth.ts` espone `setLoginSession(res, { id, email, role })` che:
    1. Firma un JWT con `jsonwebtoken` (secret `process.env.JWT_SECRET`)
    2. Serializza in cookie HTTP-only `auth=<token>; Path=/; SameSite=Lax; Max-Age=604800`
- **Lettura e validazione**
  - Ogni API protetta invoca `getPayloadFromReq(req)` per parsare il cookie, `jwt.verify` e restituisce il payload come `{ id, email, role }`
- **Ruoli**

- `authorizeRole(role, handler)` e `authorizeAdmin(handler)` controllano che `payload.role` corrisponda

### 2.2.5 Algoritmi usati

- Scroll snapping con wheel debouncing (in `Layout.tsx` `useEffect`): blocca eventi `wheel` per 500 ms e scorre di una sezione per volta.

ts

```
let ticking=false;
```

```
const onWheel=(e:WheelEvent)=>{...}
```

- `IntersectionObserver` (in `ScrollNav.tsx`): osserva le sezioni per evidenziare il dot attivo.

ts

```
const observer=new IntersectionObserver(callback,{ root: main, threshold: 0.5 });
```

(Segue codice completo, già presente in `/components/ui/ScrollNav.tsx`.)

### 2.2.6 Tailwind

---

Questa architettura garantisce un flusso chiaro:

1. **Utente/Admin** naviga sul **Frontend Next.js**
2. Quando serve, il client chiama le **API REST**
3. Le API interagiscono con **Prisma** sul **DB PostgreSQL**
4. La **sessione** rimane stateless grazie al **cookie JWT**, eliminando la necessità di memorizzare sessioni in-memory lato server.

## 3. Requisiti di sistema

### 3. Requisiti di sistema

Questo capitolo descrive le componenti software e le librerie necessarie per installare, eseguire e sviluppare correttamente il progetto “Artigianato Online”.

---

#### 3.1 Software di base

**Nota:** per lavorare in locale è consigliabile usare [nvm](#) per gestire versioni multiple di Node.js.

---

#### 3.2 Dipendenze di progetto

Di seguito l’elenco delle principali librerie e framework utilizzate, raggruppate per categoria.

##### Framework e strumenti principali

- **Next.js** ([next](#))
  - Versione: [^15.3.3](#)

| Componente          | Versione minima consigliata | Scopo nel progetto                                                                                                                                         |
|---------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Node.js</b>      | $\geq 18.x$                 | Ambiente di runtime JavaScript/TypeScript: Next.js, Prisma CLI, script di sviluppo (tailwindcss, build).                                                   |
| <b>PostgreSQL</b>   | $\geq 13.x$                 | Database relazionale principale, contiene tabelle <a href="#">users</a> , <a href="#">products</a> , <a href="#">orders</a> , etc. Gestito via Prisma ORM. |
| <b>Git</b>          | Qualsiasi $\geq 2.x$        | Controllo di versione: storicizzazione del codice, branching, collaborazioni via GitHub/GitLab.                                                            |
| <b>npm</b> (o yarn) | $\geq 8.x$                  | Package manager per installare dipendenze e lanciare script definiti in <a href="#">package.json</a> .                                                     |

- Sistema di routing basato su filesystem ([pages/](#)), rendering ibrido (SSR/SSG/ISR), API routes integrate.
- Tutto il frontend ([.tsx](#)) e il backend leggero ([pages/api/\\*.ts](#)) girano dentro Next.js.
- **Prisma** ([@prisma/client](#), [prisma](#))
  - Versione CLI: [^6.9.0](#)
  - ORM tipizzato per TypeScript/Node.js: definizione [schema.prisma](#), generazione client per accesso a PostgreSQL, migrazioni e seeding.
- **Tailwind CSS** ([tailwindcss](#), [postcss](#), [autoprefixer](#))
  - Versione: [^4.1.10](#)
  - Utility-first CSS framework per styling rapido e reattivo.
  - Configurato in [styles/globals.css](#) e integrato via PostCSS.

## Autenticazione e sicurezza

- **jsonwebtoken**
  - Versione: [^9.0.0](#)
  - Generazione e verifica di token JWT utilizzati per gestire la sessione lato cookie HTTP-only.
- **cookie**
  - Versione: [^1.0.2](#)
  - Parsing e serializzazione dei cookie (setting del JWT nel response header).
- **bcryptjs** (oppure [bcrypt](#))
  - Versione: [^2.4.3](#)
  - Hashing e confronto sicuro delle password utente.
- **next-auth**
  - Versione: [^4.24.11](#)

- (Opzionale o in parte utilizzato se si estendono provider OAuth; nel progetto abbiamo in gran parte gestito noi token custom).

## **UI, animazioni e utilità React**

- **react / react-dom**

- Versione: 18.2.0
- Libreria di base per componenti, hooks, rendering.

- **react-icons**

- Versione: ^5.5.0
- Set di icone (FiEye, FiPlus, FiTrash2, ecc.) usate in pulsanti e form.

- **framer-motion**

- Versione: ^12.16.0
- Animazioni fluide per fade-in/fade-out delle sidebar, transizioni di login, comparsa di moduli di modifica prodotti/utenti.

- **clsx**

- Versione: ^2.1.1
- Utility per concatenazione condizionale di classi Tailwind negli elementi con transizioni dinamiche.

## **HTTP client e helpers**

- **axios**

- Versione: ^1.4.0
- Richieste HTTP da frontend verso le API interne (/api/auth/login, /api/product, ecc.) con gestione credentials: 'include'.

- **tailwind-scrollbar-hide**

- Versione: ^4.0.0

- Plugin per nascondere la scrollbar di sistema in pagine o componenti specifici (admin panel).

---

### Dev-Dependencies aggiuntive

- **@types/** per TypeScript (es. [@types/node](#), [@types/react](#), [@types/jsonwebtoken](#), ecc.)
  - Consentono l'auto-completamento e i controlli di tipo in editor/IDE.
- **ts-node**
  - Versione: [^10.9.2](#)
  - Esecuzione di script TypeScript lato CLI (usato per [prisma db seed](#)).
- **prisma**
  - CLI per gestire [migrate](#), [db seed](#), [generate](#).

---

### Come verificare/installare

- **Node.js & npm:**

`node --version` # deve risultare  $\geq 18.x$

`npm --version` #  $\geq 8.x$

**PostgreSQL:** avviare un'istanza locale o remota e configurare [DATABASE\\_URL](#) in [.env](#).

- **Dipendenze:**

`npm install`

`npm run prisma db seed` # popola DB di esempio

`npm run dev` # avvia Next.js in sviluppo

- **CSS build** (se non integrato via plugin Next.js):

```
npm run build:css
```

```
npm run watch:css
```

Questo set di requisiti garantisce un ambiente omogeneo tra sviluppo e produzione, con sicurezza (JWT, bcrypt), performance (Next.js SSR/SSG, Prisma), e un design reattivo ed estendibile (Tailwind, Framer-Motion).

## 4. Installazione e configurazione

### 4. Installazione e configurazione

Questa sezione guida lo sviluppatore dal primo clone del codice fino al progetto “pronto all’uso” in locale, coprendo repository, variabili d’ambiente, dipendenze, database e CSS.

---

#### 4.1 Clonazione del repository

1. **Apri un terminale** in una cartella a tua scelta.
2. Esegui il comando:

```
git clone https://github.com/tuo-org/artigianato-online-fullstack.git
```

3. Entra nella directory del progetto:

```
cd artigianato-online-fullstack
```

4. (Opzionale) crea un branch di lavoro:

```
git checkout -b feature/il-mio-task
```

---

## 4.2 Variabili d'ambiente (.env)

All'interno della root del progetto, crea (o rinomina) il file `.env.local`:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:PORT/NOME_DB?schema=public"
```

```
JWT_SECRET="una-chiave-lunga-e-sicura"
```

```
NEXT_PUBLIC_API_BASE="/api"
```

- `DATABASE_URL` → stringa di connessione PostgreSQL usata da Prisma.
- `JWT_SECRET` → chiave segreta per firmare i token JWT (cookie HTTP-Only).
- `NEXT_PUBLIC_API_BASE` → (facoltativo) prefisso per le API.

⚠ **Non** committare mai credenziali sensibili in Git. Aggiungi `.env.local` a `.gitignore`.

---

## 4.3 Installazione dipendenze

Nel terminale, esegui:

```
bash
```

```
npm install
```

Questo installa tutte le librerie elencate in `dependencies` e `devDependencies` (`next`, `react`, `prisma`, `tailwindcss`, ecc.).

---

## 4.4 Inizializzazione Prisma e migrazioni

**Genera il client Prisma** (ogni volta che cambi `schema.prisma`):

```
bash
```

```
npx prisma generate
```

1. **Crea e applica le migrazioni** al database:

```
npx prisma migrate dev --name init
```

2. Se non hai ancora un database, questo comando lo crea basandosi su `DATABASE_URL`.

**Popola dati di test/seeding** (es. utenti fittizi, prodotti):

```
npx prisma db seed
```

3. Lo script di seeding (`prisma/seed.ts`) inserisce record demo in tabelle `users`, `products`, ecc.

---

## 4.5 Generazione CSS con Tailwind

Il progetto usa Tailwind CSS via PostCSS, con Comandi npm:

**Build una tantum** (produce `styles/tailwind.css`):

```
npm run build:css
```

**Watch in sviluppo** (ricompila ad ogni modifica):

```
npm run watch:css
```

Assicurati di aver installato `tailwindcss`, `postcss` e `autoprefixer` come dev-dependency (`npm install -D tailwindcss postcss autoprefixer`).

Se usi Next.js 13+ con plugin ufficiale, potresti integrare Tailwind direttamente in `next.config.js` e non dover eseguire manualmente il build CSS.

---

### A questo punto

- Il database è pronto,
- le dipendenze sono installate,
- il CSS generato,

le API e il frontend possono essere avviati con:

```
npm run dev
```

Apri <http://localhost:3000> e verifica il corretto funzionamento di login, visualizzazione prodotti/utenti e dashboard admin.

## 5. Struttura del progetto

### 5. Struttura del progetto e convenzioni

Questa sezione descrive l'organizzazione dei file e delle cartelle, con le principali entità del codice e le convenzioni di naming adottate in questo progetto.

---

#### 5.1 Cartelle e file chiave

```

/pages
  /api
    /auth
      login.ts      ← endpoint POST /api/auth/login
      logout.ts     ← endpoint POST /api/auth/logout
      me.ts         ← endpoint GET /api/auth/me
      users.ts      ← endpoint GET /api/auth/users
      users/[id].ts ← endpoint PUT/DELETE /api/auth/users/:id
    /product
      [id].ts       ← endpoint GET/PUT/DELETE /api/product/:id
      index.ts      ← endpoint GET /api/product
    /admin
      index.tsx     ← dashboard Admin (UI React/TSX)
    auth
      login.tsx     ← form di login utente
      register.tsx  ← form di registrazione
      _app.tsx      ← wrapper globale (Providers, Layout)
  /components
    /ui
      SidebarMenu.tsx ← menu laterale
      ScrollNav.tsx   ← "bullet points" di navigazione
      HeroSection.tsx ← sezione hero in home
      ShelfSection.tsx ← sezione scaffale (lista prodotti)
      AuthProductCard.tsx ← card prodotto per utente autenticato
      Layout.tsx      ← layout principale con navbar e scroll
  /contexts
    AuthContext.tsx  ← gestione stato di autenticazione
    CartContext.tsx  ← carrello lato client
  /utils
    auth.ts          ← JWT cookie, setLoginSession, authorize
    prisma.ts        ← istanza PrismaClient
    validators.ts     ← funzioni di validazione input (opzionale)
  /styles
    globals.css      ← file Tailwind "base" e custom CSS
    tailwind.css     ← output CSS compilato
  /prisma
    schema.prisma    ← definizione modelli e datasource
    seed.ts          ← script di popolamento ↓ i di test

```

- **/pages**: qui vivono sia le API (route-based) che le pagine React.
- **/components**: componenti UI riutilizzabili, suddivisi in "ui" (atomi/piccoli) e "root" (Layout).
- **/contexts**: React context per stato globale (autenticazione, carrello).
- **/utils**: helper e wrapper (Prisma client, JWT, validazioni).

- **/styles**: configurazione e output di Tailwind CSS.
  - **/prisma**: schema e seed del database.
- 

## 5.2 Convenzioni di naming e organizzazione

### 1. Estensione dei file

- **.ts** → moduli backend, API route, utils, context (solo logica, nessun JSX).
- **.tsx** → componenti React e pagine che contengono JSX/TSX.

### 2. Cartelle per dominio

- **/pages/api/...** → tutte le API REST, una cartella per ambito (es. **auth**, **product**).
- **/pages/<area>/...** → pagine distinte per aree funzionali (es. **admin**, **auth**).

### 3. Componenti UI

- In **/components/ui**, i singoli “atomi” e “molecole” di interfaccia (pulsanti, card, navbar).
- Prefisso sempre PascalCase: **HeroSection.tsx**, **ScrollNav.tsx**, **AuthProductCard.tsx**.

### 4. Context e utilità

- **AuthContext.tsx**, **CartContext.tsx** in **/contexts**: mai mescolarli con componenti o API.
- **/utils** raccoglie solo funzioni pure e istanze condivise (**prisma.ts**, **auth.ts**).

### 5. Naming API routes

- File corrispondente all’endpoint:
  - GET **/api/product** → **/pages/api/product/index.ts**
  - GET **/api/product/:id** → **/pages/api/product/[id].ts**
- I metodi HTTP multipli in uno stesso file (es. GET, PUT, DELETE) gestiti con **if (req.method === '...')**.

### 6. Stile CSS

- Globali e Reset in `globals.css`.
- Output Tailwind in `tailwind.css` incluso da `_app.tsx`.
- Classi Tailwind inline preferite a CSS custom, tranne per animazioni complesse (definite in `globals.css`).

## 7. Naming delle variabili e delle funzioni

- **camelCase** per variabili e funzioni (`handleSubmit`, `setLoginSession`).
- **PascalCase** per componenti e tipi (`AdminPage`, `Product`, `AuthContextType`).

Seguendo queste convenzioni, il progetto rimane **coerente**, **scalabile** e facilmente **navigabile** da qualsiasi sviluppatore che entri nel repository.

# 6. Database

## 6. Database

Questa sezione descrive in dettaglio l'architettura del database del progetto, come è modellato lo schema con Prisma, come vengono generate le migrazioni e infine come il seed script popola i dati iniziali.

---

### 6.1 Modello dati (schema Prisma)

Nel file `prisma/schema.prisma` definiamo:

#### Datasource e Generator

```
prisma

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
```

```
provider = "prisma-client-js"
}
```

- - `DATABASE_URL` è la stringa di connessione PostgreSQL (es. `postgresql://user:pass@localhost:5432/dbname`).

## Modelli principali

prisma

```
model User {
  id      String  @id @default(uuid())
  email   String  @unique
  password String
  role    Role    @default(CUSTOMER)
  firstName String @default("")
  lastName String @default("")
  cartItems CartItem[]
  orders  Order[]
  @@map("users")
}
```

```
model Product {
  id      String  @id @default(uuid())
  name    String
  description String?
  price   Float
  imageUrl String?
  createdAt DateTime @default(now())
}
```

```

    updatedAt DateTime @updatedAt

    cartItems CartItem[]

    orderItems OrderItem[]

    @@map("products")
}

```

```

model CartItem { ... }

model Order { ... }

model OrderItem { ... }

model Category { ... }

```

- 

## Enum

prisma

```

enum Role {

    CUSTOMER @map("USER")

    ARTISAN

}

```

```

enum OrderStatus {

    PENDING

    PAID

    SHIPPED

    CANCELLED

}

```

- 

- **Role** distingue clienti (“USER”) e artigiani.

- `OrderStatus` regola il flusso di un ordine.

Ogni modello Prisma genera in automatico il corrispondente TypeScript typing e le query client tramite `@prisma/client`.

---

## 6.2 Script di seed

Per popolare il database con dati di esempio (utenti di test, categorie, prodotti):

- **File:** `prisma/seed.ts`

**Comando** in `package.json`:

```
"scripts": {  
  "db:seed": "prisma db seed"  
}
```

- **Contenuto tipico:**

```
import { PrismaClient } from "@prisma/client";  
  
const prisma = new PrismaClient();  
  
async function main() {  
  // Creazione di utenti di test  
  
  const user1 = await prisma.user.create({ data: {  
    email: "mario@example.com",  
    password: "...hashed...",  
    role: "CUSTOMER",  
    firstName: "Mario",  
    lastName: "Rossi",  
  }});
```

```
// Creazione di categorie

const cat = await prisma.category.create({ data: {

  name: "Vasi",

  slug: "vasi"

}});

// Creazione di prodotti

await prisma.product.createMany({ data: [

  { name: "Vaso Leopardi", price: 49.9, categoryId: cat.id },

  { name: "Ciotola Ceramica", price: 29.5, categoryId: cat.id },

] });

}

main()

.catch(e => console.error(e))

.finally(() => prisma.$disconnect());
```

- Il comando `npm run db:seed` lancerà questo script, consentendo di avere subito un dataset di prova coerente con il front-end (es. scaffali pieni).

---

## 6.3 Migrazioni

Le migrazioni gestiscono l'evoluzione dello schema in produzione o in sviluppo:

### 1. Creazione di una nuova migrazione

```
npx prisma migrate dev --name <descrizione>
```

- Genera sotto `prisma/migrations/` la cartella della nuova migration SQL.
- Applica immediatamente le modifiche al database di sviluppo.

## 2. Applicazione in ambiente CI/CD

In produzione, si esegue:

```
npx prisma migrate deploy
```

per sincronizzare lo schema del database con tutte le migrazioni versionate.

## 3. Generazione del Prisma Client aggiornato

Dopo ogni migrazione, Prisma rigenera automaticamente il client TypeScript con i nuovi tipi, ma può anche essere forzato manualmente:

```
npx prisma generate
```

---

In sintesi, il progetto usa Prisma per:

- **Definire** in modo dichiarativo il modello dati in `schema.prisma`.
- **Versionare** lo schema tramite migrazioni SQL.
- **Popolare** con dati di esempio tramite uno script di seed.
- **Interrogare** in modo tipizzato e sicuro il database PostgreSQL da API e logica di business.

# 7. Autenticazione e autorizzazione

## 7. Sicurezza e Autenticazione

In questa sezione analizziamo in dettaglio come funziona il login sia “standard” che “master-admin”, come vengono creati e gestiti i cookie JWT, e quali middleware Prisma/Next.js proteggono le nostre API e pagine di amministrazione.

### Alternative: NextAuth e motivazioni della scelta custom

#### .1 Che cos'è NextAuth.js

NextAuth.js è una libreria open-source per Next.js pensata per semplificare la gestione delle sessioni utente e delle strategie di autenticazione.

- **Endpoint predefiniti**

NextAuth espone automaticamente `/api/auth/[...nextauth]`, con sotto-rotte per login, logout, callback OAuth, gestione sessione.

- **Provider**

Supporta out-of-the-box OAuth (Google, GitHub, Apple...), email magic-link, credenziali custom.

- **Hook React**

Offre `useSession()` per accedere in modo reattivo alla sessione in qualsiasi componente, insieme a `getSession()/getToken()` lato server.

- **Vantaggi**

- Meno boilerplate: la maggior parte della logica è già scritta.
- Aggiornamenti e integrazioni immediate con provider esterni.
- Convenzioni consolidate in community Next.js.

## **.2 Perché non è stato usato NextAuth, ma una soluzione custom**

Nel nostro progetto abbiamo deciso di gestire manualmente login, sessione e autorizzazione per avere:

### **1. Massima flessibilità sul payload JWT**

- Possiamo includere un “master password” per l’admin che abilita accesso privilegiato senza alterare la tabella utenti.
- Il payload (`id`, `email`, `role`) è esattamente quello di cui abbiamo bisogno, senza campi extra o configurazioni superflue.

### **2. Controllo totale sui cookie HTTP-only**

- Usiamo funzioni `setLoginSession(res, payload)` e `getPayloadFromReq(req)` per firmare/verificare il JWT esattamente con le policy di sicurezza (httpOnly, sameSite, secure in produzione).
- Possiamo decidere durata del token (7 giorni), nome del cookie (`auth`), e strategie di rinnovo.

### **3. Middleware personalizzati**

- E' stato scritto `authorizeRole(role, handler)` e `authorizeAdmin(handler)` per incapsulare logica di controllo accessi in pochi rigi, senza dover dipendere da hook o callback NextAuth.
- Questo approccio “bare-metal” si integra perfettamente con Next.js API Routes e Prisma.

#### 4. Riduzione delle dipendenze

- E' stato evitato di aggiungere un intero framework esterno (NextAuth.js + suoi plugin) per una feature di cui necessitavamo di una parte limitata.
- Meno dipendenze → bundle più leggero, minor impatto sugli eventuali aggiornamenti di sicurezza.

#### 5. Logging e debugging su misura

- Possiamo intercettare ogni fase di login (master, bcrypt, errori) e loggare esattamente i dettagli che ci servono (`console.debug(' [DEBUG login] ')`).

---

## 7.1 Flusso di login standard vs. master-admin

### 1. Login standard

- Il client richiama `POST /api/auth/login` con `{ email, password }`.
- Nel backend (`pages/api/auth/login.ts`):
  1. Prisma cerca l'utente via `prisma.user.findUnique({ where: { email } })`.
  2. La password fornita viene confrontata con quella salvata in `user.password` tramite `bcrypt.compare`.
  3. Se valida, si chiama `setLoginSession(res, payload)` con il payload `{ id, email, role }`.
  4. Il server restituisce `200 { message: "Login riuscito" }`.
- Il frontend, su esito OK, reindirizza l'utente alla home o alla propria dashboard.

### 2. Login master-admin

- Usando password fissa (`MASTER_PASSWORD = "12345Aa!"`), il client include un flag aggiuntivo `role: "admin"` nella richiesta.
  - Nel backend:
    1. Viene bypassato il controllo bcrypt se la password è la master.
    2. Si crea comunque il JWT, ma con `role: "ADMIN"` nel payload: così il token indica un utente “amministratore”.
    3. Nessun dato sensibile (come la password hashata) attraversa la rete.
  - Il frontend, al successo, reindirizza sempre a `/admin` e NON esegue la fetch di `/api/auth/me`, isolando la sessione come “solo admin”.
- 

## 7.2 Cookie JWT e `setLoginSession`

- **JWT (JSON Web Token)**

Nel file `utils/auth.ts`, `setLoginSession(res, payload)` genera un token firmato con `jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' })`.

- **Cookie HTTP-Only**

Il token viene serializzato in un cookie `auth` (`serialize('auth', token, { httpOnly: true, secure:..., sameSite: 'lax', path: '/', maxAge:... })`).

- **httpOnly** impedisce l'accesso via JavaScript lato client.
- **secure** è `true` in produzione (HTTPS).
- **sameSite=lax** protegge da CSRF di prima parte.

- **Recupero payload**

Ogni API protetta chiama `getPayloadFromReq(req)`, che parse del cookie, verifica il token con `jwt.verify` e restituisce il payload tipizzato `{ id, email, role }`.

---

## 7.3 Middleware `authorizeRole`

Definito in `utils/auth.ts`, questo wrapper di una API route ha firma:

```
export function authorizeRole(
```

```
role: string,  
  
handler: (req, res) => any  
  
) { ... }
```

- **Funzionamento**

1. Estrapola il payload con `getPayloadFromReq`.
2. Confronta `payload.role.toUpperCase()` con `role.toUpperCase()`.
3. Se non corrispondono, risponde 403 { message: 'Accesso negato: solo \${role}' }.
4. Altrimenti invoca il `handler` originale.

---

## 7.4 Protezione delle rotte (**authorizeAdmin** vs. **authorizeRole**)

1. **authorizeAdmin**

- Versione specializzata che fissa `role = 'ADMIN'`.
- Tutte le API di amministrazione (`/api/auth/users.ts`, `/api/product/*`, ecc.) sono esportate come:

```
export default authorizeAdmin(handler)
```

- Garantisce che solo chi ha fatto login “master-admin” (o un utente con `role: 'ADMIN'` nel DB) possa chiamarle.

2. **Pagine Next.js**

- Nelle pagine React sotto `/pages/admin/*` usiamo un context (`AuthContext`) che legge `/api/auth/me` e imposta `user.role`.
- Il componente `AdminPage` verifica `user?.role === 'ADMIN'` e, in mancanza, mostra “Accesso negato.” o reindirizza.

3. **Differenza chiave**

- **Admin:** nessuna sessione utente “standard” viene caricata, cookie JWT ha **role: 'ADMIN'**.
- **Utente normale/artigiano:** dopo il login “standard”, **role** rimane **'CUSTOMER'** o **'ARTISAN'**, e le rotte admin rifiutano l’accesso.

---

In breve, grazie a un mix di:

- **Master-password** per elevare qualunque account a “admin”
- **JWT** in cookie httpOnly
- **Middleware** che ispezionano **role**
- **Controlli frontend** in **AuthContext** e componenti

garantiamo un isolamento netto tra amministratori e utenti normali, evitando escalation di privilegi o accesso non autorizzato alle API sensibili.

## 8. API Endpoints

### 8. API Endpoints Dettagliati

Di seguito descriviamo in dettaglio ciascuna rotta API del progetto, con i relativi metodi HTTP, payload di richiesta e formato di risposta.

---

#### 8.1 POST /api/auth/login

Effettua il login sia “standard” che “master-admin”.

- **Request**

```
// per login standard
```

```
{ "email": "utente@example.com", "password": "laSuaPwd" }
```

```
// per master-admin
```

```
{ "email": "utente@example.com", "password": "12345Aa!", "role": "admin" }
```

- **Behavior**

- Cerca l'utente in Prisma (`user.findUnique({ where: { email } })`).
- Se `password === MASTER_PASSWORD && role === "admin"`, crea JWT con `role: "ADMIN"`.
- Altrimenti valida con `bcrypt.compare`; se ok crea JWT con `role` preso dal DB.
- Imposta cookie `auth` via `setLoginSession(res, payload)`.

- **Response 200 OK** { "message": "Login riuscito" }

- **401 / 403 / 400** { "message": "Accesso negato: password errata" }

---

## 8.2 POST /api/auth/logout

Chiude la sessione rimuovendo il cookie.

- **Request**

Nessun body. Solo header `credentials: include`.

- **Behavior**

Invia `Set-Cookie: auth=; Max-Age=0; Path=/; HttpOnly`.

- **Response**

- **200 OK** con corpo vuoto.

---

## 8.3 GET /api/auth/me

Restituisce i dati dell'utente loggato (dal token JWT).

- **Request**  
Cookie `auth` inviato automaticamente (`credentials: include`).
- **Behavior**
  - `getPayloadFromReq(req)` estrae e verifica JWT.
  - Se valido, risponde con `{ user: { id, email, role, firstName, lastName } }`.
- **Response 200 OK**

```
{
  "user": {
    "id": "uuid-utente",
    "email": "utente@example.com",
    "role": "CUSTOMER",
    "firstName": "Mario",
    "lastName": "Rossi"
  }
}
```

- **401 Unauthorized** `{ "message": "Non autenticato" }`

---

## 8.4 GET|PUT|DELETE /api/auth/users[:id]

Gestione CRUD utenti (protetta `authorizeAdmin`).

- **GET /api/auth/users**

### Response 200

```
[
  { "id": "uuid1", "email": "a@b.com", "role": "CUSTOMER" },
  { "id": "uuid2", "email": "c@d.com", "role": "ARTISAN" }
```

]

- **PUT /api/auth/users/:id**

#### Request

```
{ "firstName":"Luigi", "lastName":"Verdi", "email":"new@b.com", "role":"ARTISAN" }
```

○

#### Response 200

```
{ "id":"uuid1", "email":"new@b.com", "role":"ARTISAN", "firstName":"Luigi", "lastName":"Verdi" }
```

- **DELETE /api/auth/users/:id**
  - **Response 204 No Content**

---

## 8.5 GET|POST|PUT|DELETE /api/product[/:id]

CRUD Prodotti (protetta `authorizeAdmin`).

- **GET /api/product**

#### Response 200

[

```
{ "id":"pid1", "name":"Vaso Leopardi", "description":"...", "price":49.99, "imageUrl":"...png" },
```

```
{ "id":"pid2", "name":"Lampada", "description":"...", "price":29.00, "imageUrl":null }
```

]

- **POST /api/product**

## Request

```
{ "name":"Nuovo","description":"...","price":19.5,"imageUrl":null }
```

○

## Response 201

```
{ "id":"pid3","name":"Nuovo", ... }
```

○

- **GET /api/product/:id**

## Response 200

```
{ "id":"pid1","name":"Vaso Leopardi", ... }
```

○

## 404 Not Found

```
{ "message":"Prodotto non trovato" }
```

- **PUT /api/product/:id**

## Request

```
{ "price": 54.00 }
```

## Response 200

```
{ "id":"pid1","price":54.00, ... }
```

- **DELETE /api/product/:id**

○ **Response 204 No Content**

---

## 8.6 Esempi di request/response

### Login master-admin

```
curl -i -X POST http://localhost:3000/api/auth/login \  
-H "Content-Type:application/json" \  
-d '{"email":"matticio.gdl@gmail.com","password":"12345Aa!","role":"admin"}'
```

#### Response

HTTP/1.1 200 OK

Set-Cookie: auth=eyJ...; HttpOnly; Path=/; ...

```
{"message":"Login admin riuscito"}
```

#### 1. Fetch utenti (admin)

```
curl -i http://localhost:3000/api/auth/users \  
--cookie "auth=eyJ..."
```

#### Response

HTTP/1.1 200 OK

```
[ { "id": "...", "email":"...", "role":"CUSTOMER" }, ... ]
```

#### 2. Aggiorna prodotto

```
curl -i -X PUT http://localhost:3000/api/product/pid1 \  
-H "Content-Type:application/json" \  
-d '{"price":59.99}' \  
--cookie "auth=eyJ..."
```

#### 3. Response

```
{ "id":"pid1", "name":"Vaso Leopardi", "price":59.99, ... }
```

---

Con queste API ben strutturate e protette da middleware Mirage-style, il vostro **Admin Panel** e il **frontend Next.js** possono interagire in sicurezza con il database PostgreSQL tramite Prisma, garantendo chiarezza e manutenibilità.

## 9. Frontend

### 9. Frontend

Il fronte-end del progetto è sviluppato con Next.js e TypeScript, sfruttando componenti riutilizzabili e una chiara separazione tra pagine pubbliche, protette e area admin.

---

#### 9.1 Layout generale (**components/Layout.tsx**)

- **Struttura a griglia flessibile:** il componente **Layout** avvolge ogni pagina, definendo un contenitore `<div className="layout relative flex flex-col h-screen overflow-hidden">` che occupa tutta l'altezza dello schermo e gestisce lo scroll.
- **Hamburger menu:** in alto a destra c'è un pulsante fisso per aprire/chiudere la sidebar (**SidebarMenu**), con stato `menuOpen` e classe Tailwind per trasparenza e hover.
- **Main scrollato:** l'elemento `<main>` ospita la sezione “hero” e le varie sezioni “scaffale” grazie a `snap-y snap-mandatory`, consentendo uno scroll a scatti sulle sezioni, ma è disabilitato nell'area admin per scroll tradizionale.

---

#### 9.2 **SidebarMenu** e comportamento “ospite vs. admin”

- **Menu dinamico:** **SidebarMenu** riceve props `isAuthenticated`, `isArtisan`, `isAdmin` e mostra link diversi a seconda dello stato utente.
- **Admin isolato:** quando l'utente effettua il “master-login” con ruolo virtuale `ADMIN`, possiamo forzatamente `isAuthenticated=false` e `isAdmin=false` al menu: così l'admin vede solo le voci pubbliche (ospite) e non la navbar cliente/artigiano.

---

#### 9.3 **ScrollNav** e navigazione a “bullet-point”

- **Indicatore sezione attiva:** `ScrollNav` crea tanti “dot” quanti sono gli scaffali; tramite `IntersectionObserver` segnala quale sezione è in vista, ingrandendo il cerchietto corrispondente.
  - **Click sui dot:** cliccando un dot si esegue `element.scrollToView({ behavior: 'smooth' })`, che porta dolcemente alla sezione.
  - **Visibilità condizionata:** mostriamo `ScrollNav` solo sulle pagine front-end (non su `/admin` né su `/auth/login`) verificando `router.pathname`.
- 

## 9.4 Pagine pubbliche vs. protette vs. `/admin`

- **Pagine pubbliche:** homepage e pagine di categoria (`/scaffale/[slug]`) accessibili a chiunque, con `Layout` standard e `ScrollNav`.
  - **Pagine protette:** percorsi utente (es. `/cart`, `/orders`) richiedono autenticazione: il context `useAuth` carica `user` da `/api/auth/me` e, se non autenticato, reindirizza a `/auth/login`.
  - **Area admin:** tutte le pagine sotto `/admin` bypassano il layout pubblico: usano `SidebarMenu` da ospite, scroll tradizionale e visualizzano solo il pannello di amministrazione.
- 

## 9.5 Admin Panel: gestione utenti e prodotti

- **Componente principale:** `pages/admin/index.tsx` mostra due sezioni — “Utenti” e “Prodotti” — ottenendo dati da `/api/auth/users` e `/api/product`.
- **Editing inline:** cliccando “Modifica” su un elemento appare un form inline (nome, descrizione, prezzo, ecc.), pre-riempito con i placeholder dei valori attuali; bottoni `+` e `-` permettono di regolare il prezzo, e si può digitare da tastiera.
- **Operazioni CRUD:**
  - `PUT /api/auth/users/:id` aggiorna un utente;
  - `DELETE /api/auth/users/:id` lo elimina;
  - analogamente per i prodotti con `/api/product/:id`.
- **Animazioni:** l’espansione del form usa `transition-all duration-500 ease-in-out` per un effetto minimal ed elegante.

---

## 9.6 Login Page (utente e admin)

- **Login utente** (`pages/auth/login.tsx`): form con campo email e password, eye-toggle (`FiEye`, `FiEyeOff`), restyling Tailwind e fetching a `/api/auth/login`; in caso di successo reindirizza alla home.
- **Login admin** (`pages/admin-login.tsx` o simile): layout grafico simile ma il fetch invia solo email e master-password; al successo rimane isolato nel pannello admin.
- **Logout forzato**: entrambi i login page, all'`useEffect` di mount, invocano `/api/auth/logout` per azzerare sessioni residue.

---

Con questa architettura il front-end fornisce un'esperienza fluida sia per gli utenti comuni sia per l'amministratore, sfruttando pienamente le potenzialità di Next.js, React Context e Tailwind CSS.

# 10. Styling

## 10. Styling

Lo styling dell'applicazione si basa su Tailwind CSS, con alcune estensioni e utility custom per ottenere l'estetica minimal, fluida e uniforme di questo progetto.

---

### 10.1 Configurazione Tailwind

- **tailwind.config.js**
  - **content**: includiamo tutte le pagine e componenti (`./pages/**/*.{js,ts,jsx,tsx}`, `./components/**/*.{js,ts,jsx,tsx}`) in modo che Tailwind generi soltanto le classi effettivamente usate.
  - **theme.extend**: aggiungiamo la famiglia di font "Inter" sotto la chiave `sans`, così da poter usare `font-sans` globalmente.
  - **plugins**: importiamo `tailwind-scrollbar-hide` per nascondere visivamente le scrollbar interne, senza comprometterne la funzionalità.
- **styles/globals.css**

## Import di base:

CSS

@tailwind base;

@tailwind components;

@tailwind utilities;

- 
- **Gradienti di sfondo:** definiamo `body { background: linear-gradient(135deg, #b91c1c 0%, #7f1d1d 100%); }` per un rosso degradé su tutta la UI.
- **Keyframes custom:** l'animazione `fadeInUp` viene usata per far comparire testi ed errori con uno spostamento verticale da 20 px a 0, aumentando l'opacità da 0 a 1.
- **Utility scrollbar-hide:** con regole per Firefox (`scrollbar-width: none`) e WebKit (`::-webkit-scrollbar { width: 0; height: 0; }`), rendiamo invisibili le barre di scorrimento interne a `<main>` e altre sezioni scrollabili.

---

## 10.2 Utility personalizzate

- **scrollbar-hide** (via plugin): permette di applicare `scrollbar-hide` a qualunque contenitore scrollabile, nascondendo la barra senza disabilitare lo scroll. Utilizzato nel layout admin (`overflow-y-auto scrollbar-hide`) e in altre liste.

**fade-in** (animazione personalizzata): classe CSS `.animate-fade-in` associata a:

CSS

```
.animate-fade-in {  
  
  animation: fadeInUp 0.8s ease-out forwards;  
  
}
```

- Usata per mostrare i messaggi di errore nel login admin, oppure per le transizioni delle card quando appaiono.
- **Component-level styling:** i singoli componenti UI (card, form, pulsanti) sfruttano le classi Tailwind direttamente in JSX/TSX, combinando opacità (`bg-opacity-10`), backdrop blur (`backdrop-blur-md`), bordi arrotondati (`rounded-xl`), padding e flex utilities per un design coerente.

---

## 10.3 Theme e design system

- **Palette colori:**
  - Rosso primario: `#EF4444` (Tailwind `red-500`), `#B91C1C` (`red-700`) e `#7F1D1D` (`red-900`) per gradienti e hover.
  - Bianco e grigi per testi e sfondi trasparenti (`text-white`, `text-gray-300`, `bg-white bg-opacity-90`).
- **Tipografia:**
  - **Font:** “Inter” in tutto il progetto, configurato in `tailwind.config.js`.
  - **Scale di testo:** uso di `text-4xl` per titoli principali, `text-2xl` per sezioni, `text-lg` per contenuti, `text-sm` per dettagli secondari.
- **Spaziatura e layout:**
  - Unità di spaziatura multiple di 4 (es. `p-4`, `mb-6`, `space-y-4`) per mantenere ritmo visivo uniforme.
  - Grid/flex a colonne singole (`flex`, `justify-between`, `items-center`) per elenchi e header admin.
- **Component reuse:**
  - Tutti i componenti UI (bottoni, input, card) seguono le stesse classi per bordi arrotondati (`rounded/rounded-xl`), transizioni (`transition, duration-300`), e hover states (cambi di colore con `hover:text-...` e `hover:bg-...`).
  - Le utility personalizzate e le estensioni Tailwind garantiscono uno stile centrale e facilmente mantenibile, rendendo semplice l’aggiunta o modifica di colori e animazioni in un unico file di configurazione.

## 10.4 Funzioni custom

- Middleware di autorizzazione `authorizeRole(role, handler)`

**Dove:** `/utils/auth.ts`

**Cosa fa:** Wrappa una API route Next.js in modo che solo utenti con il ruolo specificato possano accedervi.

### Come funziona:

1. Estrae il JWT dal cookie `auth` (`getPayloadFromReq`).
2. Verifica che `payload.role.toUpperCase() === role.toUpperCase()`.
3. In caso negativo ritorna `403 { message: "Accesso negato: solo ${role}" }`.
4. Altrimenti richiama il `handler` originale.

### Esempio di uso:

ts

```
// pages/api/orders.ts

import { authorizeRole } from '../utils/auth';

export default authorizeRole('ARTISAN', async (req, res) => {

  // logica solo per artigiani

});
```

- `authorizeAdmin(handler)`

**Dove:** `/utils/auth.ts`

**Cosa fa:** Versione specializzata di `authorizeRole`, fissa `role = 'ADMIN'`.

**Perché:** Serve a proteggere tutte le rotte di amministrazione (es. `/api/auth/users`, `/api/product/*`).

- Operazioni CRUD con Prisma

Nel progetto usiamo **Prisma Client** per interagire con il database PostgreSQL, grazie al suo client tipizzato:

```
prisma.user.findUnique({ where: { email } })
```

- Restituisce un singolo utente per chiave univoca.

- Esempio: nel login, recuperiamo la riga utente per verificarne la password.

**prisma.user.findMany({ select: { id, email, role } })**

- Ritorna tutti gli utenti (o un sotto-insieme di campi) per la pagina admin.
- Selettore **select** minimizza i dati trasferiti.

**prisma.user.update({ where: { id }, data: {...} })**

- Aggiorna i campi utente passati in **data**.
- Usato nelle API **PUT** /api/auth/users/[id].

**prisma.user.delete({ where: { id } })**

- Cancella un utente per **id**.
- Usato in **DELETE** /api/auth/users/[id].

Stesse chiamate esistono per **prisma.product.\*** nelle API di prodotto.

- React Icons per icone inline

**Libreria:** **react-icons/fi** (Feather Icons).

**Componenti utilizzate:**

- **FiEye** e **FiEyeOff** (toggle visibilità password).
- **FiPlus** e **FiMinus** (bottoni per incrementare/diminuire quantità o prezzo).
- **FiTrash2**, **FiEdit2**, ecc.

**Perché:**

- SVG leggero, consente styling con CSS/Tailwind (`className="text-white"`).
- Integrazione diretta in JSX senza asset esterni.

- **Gestione JWT su cookie**

*setLoginSession(res, payload)*

- **Dove:** `/utils/auth.ts`

- **Cosa fa:**

1. Crea un token JWT firmato con `JWT_SECRET`.
2. Serializza un cookie HTTP-only `auth=<token>` con durata 7 giorni.
3. Lo invia in header `Set-Cookie`.

- **Uso:** in `/api/auth/login` sia per login master-admin sia per login classico.

*getPayloadFromReq(req)*

- **Cosa fa:**

1. Legge `req.headers.cookie`.
2. Parsea i cookie e prende quello `auth`.
3. Fa `jwt.verify(token, JWT_SECRET)` per ottenere il payload.

- **Uso:** dentro `authorizeRole` e `authorizeAdmin`, ma anche in `/api/auth/me` per restituire i dati dell'utente corrente.

- **axios** per chiamate client-side

**Dove:** in tutti i componenti React (`AdminPage`, `LoginPage`, ecc.).

**Perché:**

- API promise-based, supporta error handling con `try/catch`.
- Gestione automatica di JSON request/response.

#### Esempio:

```
ts

useEffect(() => {

  axios.get('/api/auth/users').then(res => setUsers(res.data));

}, []);
```

### 10.2.5 Funzioni e librerie custom

*setLoginSession(res, payload) & getPayloadFromReq(req)*

- Scopo

Queste due funzioni si occupano di **emettere**, **leggere** e **verificare** il JSON Web Token (JWT) che gestisce la sessione utente via cookie HTTP-only.

*setLoginSession(res, payload)*

- **Cosa fa**
- Riceve un `payload` (contenente ad esempio `id`, `email`, `role`).
  - Firma un token JWT con la chiave segreta `JWT_SECRET` e impostazione di scadenza a 7 giorni.
  - Serializza il token in un cookie chiamato `auth`, con flag `httpOnly`, `secure` (in produzione), `sameSite='lax'` e `maxAge` corrispondente.
  - Aggiunge il header `Set-Cookie` alla risposta API.
- **Perché è importante**
  - Mantiene l'autenticazione lato client in modo sicuro (i JavaScript in pagina non possono leggere il cookie).
  - Permette di non dover memorizzare token in LocalStorage o sessionStorage, riducendo il rischio di XSS.

*getPayloadFromReq(req)*

- **Cosa fa**

- Legge l'header **cookie** della richiesta.
- Estrapola il cookie **auth**.
- Verifica e decodifica il JWT (usando la stessa **JWT\_SECRET**).
- Restituisce il **payload** (user id, email, role) oppure **null** se il token non c'è o è scaduto.

- **Perché è importante**

- Qualsiasi API protetta chiama questa funzione per capire “chi sta facendo la richiesta” e con quale ruolo, senza dover ogni volta interrogarne il database.

*authorizeRole(role, handler) / authorizeAdmin(handler)*

- **Scopo**

Middleware di Next.js API Routes che **filtra** l'accesso alle rotte in base al ruolo utente.

*authorizeRole(role, handler)*

- **Come funziona**

1. Estrae il payload con **getPayloadFromReq(req)**.
2. Se manca il token o **payload.role** non corrisponde al **role** atteso, risponde con **403 Accesso negato**.
3. Altrimenti invoca il **handler** vero e proprio.

- **Uso pratico**

ts

```
export default authorizeRole('ARTISAN', async (req, res) => {  
  
  // qui so che chi chiama è un artigiano
```

```
...  
})
```

*authorizeAdmin(handler)*

- **Semplificazione**

È un'alias di `authorizeRole('ADMIN', handler)`.

Viene usato per tutte le API che devono essere accessibili soltanto dall'admin master (pannello di controllo).

- **Perché è utile**

- Mantiene DRY: non si ripete ogni volta il controllo sul payload.
- Chiarezza semantica: `authorizeAdmin` indica subito che la rotta è “solo per admin”.

## **Prisma** (`prisma.user.*`, `prisma.product.*`)

- **Scopo**

Prisma Client è l'ORM tipizzato usato per interagire con PostgreSQL in modo sicuro e tipizzato.

### **Operazioni CRUD principali**

- **findUnique**

ts

```
const user = await prisma.user.findUnique({ where: { email } })
```

→ recupera un singolo record (o `null`).

- **findMany**

ts

```
const users = await prisma.user.findMany({ select: { id: true, email: true } })
```

→ lista di record, con selezione campi.

- **update**

ts

```
const updated = await prisma.product.update({  
  where: { id },  
  data: { price: newPrice }  
})
```

→ modifica un record esistente.

- **delete**

ts

```
await prisma.user.delete({ where: { id } })
```

→ rimuove un record.

Perché Prisma?

- Tipizzazione end-to-end (TypeScript): schema → client.
- Migrazioni gestite via CLI.
- Query builder pulito e leggibile, con IntelliSense.

## AXIOS

### Scopo

Libreria HTTP client-side basata su promise, usata in tutti i componenti React per comunicare con le API interne.

Caratteristiche principali

- **Intercettori**: possibilità di iniettare token o gestire errori globali.

### Sintassi semplice:

ts

```
const res = await axios.post('/api/auth/login', { email, password })
```

- 
- **Gestione automatica del JSON:** `res.data` è già l'oggetto JS.

Nel progetto

- Richieste di login, logout, CRUD di utenti/prodotti nel pannello admin.
- Aggiornamento dello stato React (`setUsers`, `setProducts`) sulla risposta.

## React-icons/fi

### Scopo

Pacchetto di icone SVG “Feather” per React, importabili come componenti.

Icone utilizzate

- **FiEye / FiEyeOff** → toggle visibilità password nei form.
- **FiPlus / FiMinus** → bottoni per incrementare/diminuire numeri (prezzo, quantity).
- **FiEdit2 / FiTrash2** → pulsanti Modifica/Elimina nel pannello admin.

### Vantaggi

- SVG ottimizzati, leggerissimi.
- Styling via `className`/Tailwind (dimensione, colore).
- Nessuna dipendenza da font esterni.

# 11. Automazioni e script

## 11.1 Script NPM (dev, build, watch:css, ...)

Nel `package.json` del progetto sono definiti i comandi di sviluppo, build e generazione automatica del CSS Tailwind:

- **npm run dev**

Avvia il server Next.js in modalità “development” sulla porta di default (3000). Ricarica automaticamente le pagine al salvataggio dei file e mostra errori in overlay.

json

`"dev": "next dev"`

*Utilizzato quotidianamente durante lo sviluppo frontend e backend.*

- **npm run build**

Esegue una compilazione ottimizzata della codebase. Next.js trasforma le pagine in bundle minificati, genera le rotte statiche quando possibile e prepara il progetto alla messa in produzione.

json

`"build": "next build"`

- **npm start**

Avvia il server Next.js in “production mode” dopo aver fatto la build. Serve le pagine pre-renderizzate e usa il middleware di Next.js in modo performante.

json

`"start": "next start"`

- **npm run lint**

Esegue Next-lint sulla codebase, controllando stile e potenziali errori di codice (sia TypeScript che React).

json

CopiaModifica

`"lint": "next lint"`

- **npm run build:css**

Invoca la CLI di Tailwind CSS per compilare le utility da `styles/globals.css` in un file CSS statico (`styles/tailwind.css`).

json

CopiaModifica

```
"build:css": "tailwindcss -i ./styles/globals.css -o ./styles/tailwind.css"
```

*Impiego puntuale quando si aggiorna `tailwind.config.js` o si aggiungono nuove classi custom.*

- **npm run watch:css**

Avvia la modalità “watch” di Tailwind: ogni modifica ai file monitorati—sia i CSS globali sia i template JSX/TSX—rigenera `tailwind.css` in tempo reale.

json

CopiaModifica

```
"watch:css": "tailwindcss -i ./styles/globals.css -o ./styles/tailwind.css --watch"
```

*Fondamentale per un workflow rapido di prototipazione delle UI.*

- **npm run prisma, npm run db:seed**

- **prisma** espone il CLI di Prisma (migrazioni, introspezione, generazione client).
- **db:seed** esegue lo script di popolamento iniziale (`prisma/seed.js`), utile su nuovi ambienti o dopo reset del database.

---

## 11.2 Automazioni eventuali (reminder, deploy, ...)

Per snellire attività ricorrenti o critiche, si possono aggiungere automazioni di vario tipo:

### 1. Deploy Continuo

- **Strumenti:** GitHub Actions, GitLab CI, o Vercel.
- **Workflow:** a ogni push sul branch `main` (o `master`), eseguire `npm ci`, `npm run build:css`, `npm run build`, infine pubblicare l’app su Vercel o su un server dedicato.
- **Vantaggio:** zero operazioni manuali, ogni modifica approvata diventa immediatamente disponibile in produzione.

### 2. Reminder per aggiornamento dipendenze

- **Descrizione:** un task settimanale che controlla la presenza di versioni più recenti delle dipendenze critiche (`next`, `prisma`, `tailwindcss`, ecc.).

- **Implementazione:** utilizzare un bot o GitHub Action con [npm outdated](#), notificando via email o Slack se è disponibile un upgrade.

### 3. Backup periodico del database

- **Frequenza:** giornaliera o settimanale.
- **Azione:** dump automatico del database PostgreSQL (es. [pg\\_dump](#)), archiviazione su cloud storage (S3, Google Cloud Storage).
- **Ripristino:** procedure documentate per il restore da file di backup.

### 4. Monitoraggio prestazioni e errori

- **Tool:** Sentry per error tracking, Datadog o New Relic per APM.
- **Setup:** integrazione del client Sentry in Next.js ([@sentry/nextjs](#)), configurazione DSN; alert via Slack o email in caso di crash o latenza anomala.

### 5. Generazione automatica di documentazione

- **Script:** usare [typedoc](#) o [JSDoc](#) per ricavare commenti e tipi da [utils/](#) e [contexts/](#).
- **Schedulazione:** mensile, produce una doc HTML interna e la pubblica su GitHub Pages o Confluence.

**Nota sul progetto:** finora abbiamo gestito manualmente i comandi NPM e le migrazioni Prisma. Integrare almeno il deploy CI/CD (ad esempio con Vercel) e un backup nightly garantirebbe maggiore stabilità e rapidità di rilascio—elementi essenziali in un contesto di e-commerce in continua evoluzione.

## 12. Testing & Quality Assurance

### 12. Testing

Una strategia di testing solida è fondamentale per garantire che ogni parte dell'applicazione funzioni correttamente, resista a refactoring e continui a supportare l'evoluzione delle funzionalità. In questo progetto — un e-commerce full-stack con Next.js, Prisma e Tailwind CSS — consigliamo di coprire tre livelli di test: unitari, di integrazione e end-to-end.

---

#### 12.1 Tipi di test

## 1. Test unitari

- **Obiettivo:** verificare il comportamento di singole funzioni, helper e componenti isolati.
- **Esempi nel progetto:**
  - Funzioni di utilità in `utils/auth.ts` (es. `getPayloadFromReq`, calcolo dell'expiry del token).
  - Parser e validatori di input (ad es. controllo delle email, validazione password).
  - Piccoli componenti React “presentazionali” in `components/ui`, come il pulsante “bullet” di `ScrollNav` o il form control con icona occhio in `LoginPage`.

## 2. Test di integrazione

- **Obiettivo:** assicurarsi che più unità (ad es. API route + database) funzionino bene insieme.
- **Esempi nel progetto:**
  - Chiamate a `/api/auth/login` contro un database di test Prisma in memoria (SQLite), controllando che:
    1. Utente registrato e password corretta → cookie JWT venga emesso.
    2. Master-password → login admin.
  - Comportamento di `authorizeRole/authorizeAdmin` middleware: risposte 200 vs. 403 in base al ruolo nel token.
  - Integrazione tra front-end React e API: il componente `AdminPage` che fetches `/api/auth/users` e `/api/product` e popola le tabelle.

## 3. Test end-to-end (E2E)

- **Obiettivo:** simulare il flusso completo dell'utente in un browser reale (o virtuale), dalla UI fino al database.
- **Esempi nel progetto:**
  - Flusso di login utente → navigazione alla home protetta, aggiunta di un prodotto al carrello, checkout simulato.

- Flusso di login admin → accesso al pannello admin, modifica di un prodotto e verifica visiva della lista aggiornata.
  - Verifica che la navigazione “bullet-point” scorra tra le sezioni Hero e scaffali in pagina pubblica.
- 

## 12.2 Strumenti consigliati

- **Jest**
  - Framework di test JavaScript/TypeScript, ottimo per test unitari e di integrazione.
  - Con `ts-jest` supporta TypeScript out-of-the-box.
  - Permette di mockare moduli (`prisma client`, `jsonwebtoken`) per isolare le funzioni da testare.
- **React Testing Library**
  - Complementare a Jest per testare componenti React dal punto di vista dell'utente (queries `getByText`, `fireEvent.click`).
  - Utile per verificare rendering condizionale di `AdminPage`, comportamento dei form e animazioni CSS (es. classi attivate in `ScrollNav`).
- **Supertest**
  - Per testare direttamente le API Next.js (`pages/api/...`).
  - Esegue richieste HTTP ai route handler, consente di controllare status code, headers, body JSON.
- **Playwright o Cypress**
  - Strumenti E2E per automatizzare browser reali.
  - Permettono di scrivere scenari (login, navigazione, modifiche dati) con screenshot e video.
  - Netta integrazione con CI: fail build se un flusso critico fallisce.
- **Testing Database**

- Utilizzare una SQLite in-memory per isolare i test Prisma, evitando side-effect sul database di sviluppo.
  - Configurare uno schema di test in `jest.setup.ts` o in un `beforeAll` globale.
- 

## 12.3 Best practices

### 1. Isolamento dei test

- Ogni test unitario o di integrazione deve poter girare indipendentemente dagli altri.
- Prima di ogni test, reset del database di test (`prisma migrate reset --preview-feature` o `prisma db push` in memoria).

### 2. Coverage minimo

- Puntare a coprire almeno il **70–80%** del codice, con focus sulle logiche di business (login, autorizzazioni, CRUD).
- Generare report di coverage con Jest (`--coverage`) e integrarli in pipeline CI.

### 3. Test di regressione su UI

- In E2E, salvare screenshot di baseline e comparare in ogni build nightly: catturare regressioni visuali su Layout, pulsanti, form.

### 4. Mock e spy con cautela

- Mockare le dipendenze esterne (es. chiamate a servizi esterni) ma **non** mockare troppo: nei test di integrazione vogliamo confermare che Prisma + Next.js API Routes dialoghino effettivamente con il database.

### 5. CI/CD integrato

- Far girare tutti i test (unitari, integrazione, E2E) ad ogni pull request. Bloccare il merge se i test falliscono o la coverage scende sotto soglia.

### 6. Testing locale veloce

Script dedicati in `package.json`:

```
json
CopiaModifica
"test": "jest --passWithNoTests",
```

"test:watch": "jest --watch",

"test:e2e": "cypress open",

"test:unit": "jest --testPathPattern=src/ --coverage"

- 
- In questo modo ogni sviluppatore può facilmente eseguire solo i test unitari o l'intera suite.

---

**In sintesi**, per il nostro progetto consigliamo di:

1. Scrivere subito i test unitari delle funzioni in `utils/` e dei piccoli componenti in `components/ui/`.
2. Aggiungere test di integrazione per tutte le API in `pages/api/`.
3. Configurare una suite E2E con Playwright o Cypress per coprire i flussi critici (login, admin CRUD, navigazione a “bullet-point”).
4. Integrare la suite in CI (es. GitHub Actions) e impostare soglie di coverage.

Questo approccio garantisce che l'e-commerce rimanga stabile, sicuro e manutenibile anche al crescere delle funzionalità.

## 13. Deployment

### 13. Deployment e operatività in produzione

Una volta completato lo sviluppo e i test sull'ambiente locale, è essenziale predisporre un flusso di rilascio robusto per portare l'applicazione in produzione. Nel contesto di questo progetto — un e-commerce full-stack con Next.js, Prisma e PostgreSQL — i punti chiave da curare sono la configurazione dell'ambiente di produzione, la scelta della piattaforma di hosting, la gestione sicura delle variabili d'ambiente e la strategia di backup del database.

---

#### 13.1 Impostazione dell'ambiente di produzione

1. **Build ottimizzata**

In produzione, Next.js deve essere compilato con:

`npm run build`

`npm run start`

- 
- Questo comando esegue l'ottimizzazione degli asset, il tree-shaking, la generazione delle pagine statiche (SSG) e l'avvio di un server Node.js che serve SSR e le API Routes.

## 2. File di configurazione

- Assicurarsi che `next.config.js` non includa flag di debug (`reactStrictMode`, `swcMinify`) non supportati in prod, e configuri eventuali rewrite o redirect per URL SEO-friendly (es. `/scaffale/:slug`).

## 3. Logging e monitoring

- Inserire middleware di logging (es. `morgan`) sulle API per tracciare gli errori 5xx/4xx.
- Integrare un servizio di monitoring come Sentry per catturare eccezioni lato server e client.

---

## 13.2 Piattaforme di hosting

### ● Vercel

- Scelta naturale per Next.js: deploy “zero-config”, preview deploy ad ogni branch, automazioni GitHub.
- Vercel gestisce automaticamente SSG, ISR (Incremental Static Regeneration) e serverless functions per le API Routes.

### ● Heroku

- Può ospitare un semplice dyno Node.js che esegue `npm run start`.
- Configurare un add-on “Heroku Postgres” per il database, con piani free/paid.

### ● DigitalOcean / AWS / Google Cloud

- Deploy su Droplet/EC2 con Docker Compose: un container Next.js, uno per PostgreSQL, un reverse-proxy Nginx.

- Consente maggiore controllo ma richiede più setup.

---

### 13.3 Variabili d'ambiente in produzione

Tutte le credenziali e le chiavi segrete devono risiedere in variabili d'ambiente, **mai** nel codice:

| Variabile                 | Descrizione                                                        |
|---------------------------|--------------------------------------------------------------------|
| <code>DATABASE_URL</code> | URI di connessione a PostgreSQL (es. <code>postgres://...</code> ) |
| <code>JWT_SECRET</code>   | Secret usato per firmare/verificare i JWT di auth                  |
| <code>NEXTAUTH_URL</code> | (se si usa next-auth) URL pubblico dell'app                        |
| <code>NODE_ENV</code>     | <code>production</code>                                            |

- - **In Vercel:** si configurano nella dashboard del progetto, “Environment Variables”.
- **In Heroku:** tramite `heroku config:set DATABASE_URL=... JWT_SECRET=....`
- **In Docker:** nel file `docker-compose.yml`, sotto `environment:`.

---

### 13.4 Backup del database

Un database di produzione richiede una policy di backup per garantire recovery in caso di guasti o cancellazioni accidentali:

1. **Backup automatico**

- Su Heroku Postgres, abilitare “Heroku PGBackups” con snapshot giornaliero.
- Su DigitalOcean Managed Database, attivare backup automatici con retention di 7–14 giorni.

## 2. Backup manuale

- Eseguire periodicamente un dump con Prisma:

```
npx prisma db pull
```

```
PGPASSWORD=$DB_PASS pg_dump -Fc --no-acl --no-owner -h $DB_HOST -U  
$DB_USER $DB_NAME > backup_$(date +%F).dump
```

## 3. Ripristino

- Tenere istruzioni chiare nel README per importare il dump in un’istanza PostgreSQL di test o staging, es.:

```
bash  
CopiaModifica  
pg_restore --verbose --clean --no-acl --no-owner -h localhost -U user -d mydb  
backup.dump
```

## 4. Verifica periodica

- Automatizzare un job (es. GitHub Actions) che carica un backup in un ambiente di staging e verifica integrità e migrazioni.

---

**In sintesi**, per un deployment production-ready:

1. Eseguire build Next.js in modalità production.
2. Scegliere una piattaforma (Vercel semplifica al massimo).
3. Gestire con cura le variabili d’ambiente, tenendole fuori dal repository.
4. Implementare backup automatici e procedure di restore documentate.

Questo garantisce un rilascio sicuro, affidabile e facilmente manutenibile anche con crescite di traffico e dati.

# 14. Manutenzione e troubleshooting

## 14. Gestione degli errori, logging e versionamento delle dipendenze

Nel corso dello sviluppo e soprattutto durante la fase di messa in produzione, è fondamentale saper riconoscere e risolvere tempestivamente i problemi più comuni, disporre di un sistema di log efficace e mantenere aggiornate le librerie senza interrompere la stabilità del progetto. Qui di seguito vediamo i casi più ricorrenti e come sono gestiti nel nostro e-commerce Next.js + Prisma.

---

### 14.1 Errori comuni

#### 1. “tailwindcss non riconosciuto come comando”

- **Causa:** Tailwind installato localmente (`devDependency`) ma non in PATH globale.
- **Soluzione:**

- Assicurarsi di avere uno script NPM:

```
json

"scripts": {

  "build:css": "tailwindcss -i ./styles/globals.css -o ./styles/tailwind.css",

  "watch:css": "tailwindcss -i ./styles/globals.css -o ./styles/tailwind.css --watch"

}
```

- Eseguire via NPM: `npm run watch:css` (non `tailwindcss ...` diretto).
- Verificare che `node_modules/.bin/tailwindcss` esista e che `devDependencies` includano `"tailwindcss": "^4.x"`.

#### 2. “Cannot read property ‘toFixed’ of undefined” su `product.price.toFixed(2)`

- **Causa:** qualche prodotto in risposta API non ha il campo `price` valorizzato.
  - **Soluzione:**
    - **Backend:** garantire sempre un valore (es. `0.0`) in Prisma, rendendo `price @default(0)` o aggiungendo validazione.
    - **Frontend:** usare guardia `product.price !== null ? product.price.toFixed(2) : '0.00'`.
3. “Metodo non consentito” (405) su chiamate API
- **Causa:** rotte Next.js `.ts` che gestiscono solo GET/PUT/DELETE e ricevono un POST.
  - **Soluzione:** verificare `if (req.method !== 'POST') return res.status(405)...` e chiamare la rotta corretta, oppure estendere il handler per supportare anche il metodo richiesto (es. `PATCH`).
4. “Unexpected token '<' ... is not valid JSON” in console NextAuth
- **Causa:** fetch verso `/api/auth/session` ritorna HTML di una pagina di errore 404 o login page.
  - **Soluzione:** controllare che la route `/api/auth/session` sia configurata (se si usa NextAuth), o rimuovere NextAuth se si adotta un sistema custom basato su JWT.
5. Rimbalzo del viewport tra sezioni con `scrollIntoView`
- **Causa:** listener `wheel` personalizzato con `e.preventDefault()` che forza lo scroll programmato e blocca lo scroll naturale.
  - **Soluzione:**
    - Limitarne l’uso solo alle pagine pubbliche (es. evitare su `/admin`).
    - Tarare il tempo di “ticking” (delay fra un comando e l’altro) per rendere più fluido l’avanzamento.
- 

## 14.2 Log e debug

### 1. Console log di debug

- Abbiamo introdotto `console.log('[DEBUG login] email:', email, ...)` solo temporaneamente per verificare payload e risposte API.

- **Rimozione:** in `pages/api/auth/login.ts`, eliminare ogni `console.log` prima del deploy.

## 2. Errori lato server

- Nei middleware `authorizeRole` e `authorizeAdmin`, ogni eccezione viene catturata e loggata su `console.error('[auth] ...', err)`.
- In produzione conviene inviare questi log a un servizio (Sentry, LogRocket) invece che in stdout.

## 3. Client-side errori

- In componenti React, gestiamo `catch` su `fetch/axios` e impostiamo uno stato `setError(...)`, che genera una UI di notifica (banner rosso al centro).
- Questo aiuta l'utente a capire se il problema è di rete, credenziali o permessi.

## 4. Level di log

- **Info:** accessi riusciti, creazione sessione (`setLoginSession`).
- **Warn:** tentativi login falliti per password errata o utente non trovato.
- **Error:** crash imprevisti, errori di sintassi JSON, indisponibilità del DB.

---

# 14.3 Aggiornamenti di dipendenze

## 1. Strategia semver

- Seguire versioni MAJOR.MINOR.PATCH:
  - **PATCH** (`^1.2.3 → 1.2.x`): correzioni bug sicure.
  - **MINOR** (`^1.2.3 → 1.x.x`): nuove feature compatibili.
  - **MAJOR** (`^1.2.3 → 2.x.x`): breaking changes → test approfonditi.

## 2. Controllo regolare

- Eseguire periodicamente `npm outdated` per vedere pacchetti da aggiornare.
- Aggiornare prima le **devDependencies** (es. Tailwind, TypeScript, eslint) e verificare che build/CSS non rompano nulla.

- Passare poi alle **dependencies** di runtime (Next.js, React, Prisma Client).

### 3. Test dopo aggiornamenti

- Ogni aggiornamento significativo richiede un ciclo di test:
  - **Manual smoke test**: login, CRUD utenti/prodotti, navigazione a bullet point.
  - **Automatizzazione**: script di test E2E con Cypress o Playwright (scorrimento, autenticazione, API).

### 4. Changelog e locking

- Mantenere un CHANGELOG.md nel repo per tenere traccia di versioni, new feature e bugfix.
- Usare `package-lock.json` per garantire che tutti usino le stesse versioni, evitando “works on my machine”.

---

In sintesi, un approccio organizzato a error handling, logging e versioning delle dipendenze garantisce che il progetto rimanga robusto, manutenibile e facilmente aggiornabile, riducendo al minimo downtime e regressioni.

## 15. Appendici

### 15.1 Glossario

- **Next.js**  
Framework React full-stack che gestisce rendering SSR/SSG e API Routes. Nel nostro progetto alimenta sia il frontend (.tsx) sia le API (.ts) sotto `/pages/api/*`. In oltre unifica frontend e API, semplifica routing, supporta SSR/SSG out-of-the-box e facilita il deployment (Vercel, Heroku).
- **Prisma**  
ORM per Node.js/TypeScript che genera un client fortemente tipizzato sulla base dello `schema.prisma`. Lo usiamo in `/utils/prisma.ts` e in tutte le API per leggere/scrivere in PostgreSQL.
- **JWT (JSON Web Token)**  
Standard per trasportare in modo sicuro informazioni (“payload”) tra client e server. Nel nostro progetto un cookie `auth` contiene un JWT firmato con `JWT_SECRET`, creato da

`setLoginSession` e validato in `getPayloadFromReq`.

- **Master-admin password**  
Password fissa (`12345Aa!`) che consente l'accesso in "modalità amministratore" a qualunque utente esistente, bypassando il controllo `bcrypt`. Gestita in `/pages/api/auth/login.ts`.
- **Middleware**  
Funzioni wrapper (`authorizeRole`, `authorizeAdmin`) applicate alle API Routes per bloccare l'accesso se il JWT non corrisponde al ruolo richiesto.
- **ScrollNav**  
Componente "bullet-point" fisso che rileva con un `IntersectionObserver` in quale "sezione" (hero / scaffali) l'utente si trova e ne evidenzia il dot corrispondente.
- **Tailwind-scrollbar-hide**  
Plugin Tailwind CSS che ci permette di nascondere la scrollbar pur mantenendo lo scrolling attivo, usato ad esempio in `/styles/globals.css` per il pannello admin.
- **ShelfSection**  
Componente UI che mostra un carosello di prodotti di una categoria; in homepage è ripetuto per ogni categoria con 3 prodotti di anteprima.
- **HeroSection**  
Sezione di apertura a pieno schermo, con call-to-action. Prima "slide" della UI a pagine multiple.

---

## 15.2 Risorse e link utili

- **Documentazione Next.js**  
<https://nextjs.org/docs>
- **Guida Prisma**  
<https://www.prisma.io/docs>
- **Tailwind CSS**  
<https://tailwindcss.com/docs>
- **JSON Web Tokens**  
<https://jwt.io/introduction/>
- **React Icons** (usato per FiEye, FiPlus ecc.)  
<https://react-icons.github.io/react-icons/>

- **Sentry** (consigliato per log):  
<https://sentry.io/welcome/>
- 

### 15.3 Contatti del team

- **Project Lead & Backend**  
Matteo Di Liberto – [mdiliberto1@studenti.uninsubria.it](mailto:mdiliberto1@studenti.uninsubria.it)
- **Frontend & UI/UX**  
Matteo Di Liberto – [mdiliberto1@studenti.uninsubria.it](mailto:mdiliberto1@studenti.uninsubria.it)
- **DevOps & Deployment**  
Matteo Di Liberto – [mdiliberto1@studenti.uninsubria.it](mailto:mdiliberto1@studenti.uninsubria.it)
- **Supporto Tecnico**  
[support@artigianato.it](mailto:support@artigianato.it)

*Per richieste di accesso, issue critiche o proposte di feature aprire un ticket sulla repository GitHub.*